

Spring 2026 COP 3502H Final Exam Part B 4/30/2026 Solutions

A1) (10 pts) DSN (Dynamic Memory Management in C)

Each farm in a group of farms has some number of crops. Each crop has a total yield, in pounds. For this question, write a function that takes in the information about all of the crops on all of the farms, and returns a single array (dynamically allocated) storing sum of pounds of all of the crops for each farm. For example, if there were 3 farms with the crop yields of {2, 5, 4}, {2, 1}, and {9, 3, 4, 5}, your function should return a single array storing {11, 3, 21}. The input to the function will be the following:

`int** allcrops` - 2D array with all crop data. `allcrops[i]` is an array which stores the separate yields for each crop on farm `i`.
`int* numcrops` - 1D array where `numcrops[i]` stores the number of different crops for farm `i`.
`int numFarms` - The number of farms for which `allcrops` stores data.

Fill in the function below so that it returns a pointer to the array storing the sum of crop yields for each farm. Do not modify the contents of either `allcrops` or `numcrops`.

```
int* totalYield(int** allcrops, int* numcrops, int numFarms) {  
  
    // Grading: 2 pts memory, 1 pt set to 0.  
    int* res = calloc(numFarms, sizeof(int));  
  
    // 1 pt.  
    for (int i=0; i<numFarms; i++)  
  
        // 2 pts.  
        for (int j=0; j<numcrops[i]; j++)  
  
            // 3 pts  
            res[i] += allcrops[i][j];  
  
    // 1 pt  
    return res;  
}
```

A2) (10 pts) DSN (Linked Lists)

Consider storing an unsigned binary number in a linked list, storing one bit in each node, where the first node stores the bit value of the least significant bit. Incrementing the value of an integer stored in this manner never requires changing where the pointer to the first node is pointing, thus we can design an increment function to take in a pointer to a linked list storing one of these integers and perform the increment without returning anything. For example, if number was pointing to the list $\rightarrow 0 \rightarrow 1$, then the increment function would change this list to look like $\rightarrow 1 \rightarrow 1$, incrementing the number from a value of 2 to 3. If we again called the increment function on the list storing 3, it would change the list to be $\rightarrow 0 \rightarrow 1$. The first node isn't changed; rather the value stored in it is flipped from 1 to 0. Same for the second node. Then, a third node is added to the list, storing 1. Complete the function below, **writing it iteratively**, to perform the increment on a binary number stored in this fashion. Use the struct given below. You may assume that number is pointing to a list storing a valid non-negative binary number, meaning that the last node will store 1 unless the one and only node in the list stores 0.

```
typedef struct node {
    int bit;
    struct node* next;
} node;

void increment(node* number) {

    node* prev = number;

    // 3 pts looping mechanism (so this and last line of loop)
    while (number != NULL) {

        // 2 pts for checking and flipping here.
        if (number->bit == 1)
            number->bit = 0;

        // 2 pts for this flip and 1 pt to correctly get out.
        else {
            number->bit = 1;
            return;
        }

        prev = number;
        number = number->next;
    }

    // 3 pts for this special case.
    prev->next = malloc(sizeof(node));
    prev->next->bit = 1;
    prev->next->next = 0;
}
```

A3) (5 pts) ALG (Queue)

In class, the breadth first search algorithm using a queue was shown to go through visiting squares in a corn maze (a 2d grid with some marked out squares) from a starting point. Assuming that the order of enqueueing squares from a current square is up, right, left and down, show the order that the following squares in the grid below get visited. The valid squares are numbered 1 to 20 and the marked out squares are shown with an x. Start the search at square number 6 (my favorite number!) (Hint: Remember that the squares get visited based on distance order, so all squares that are a distance of 2 from the starting point get visited after all squares that are a distance of 1 from the starting point.)

1	2	3	4
5	x	6	7
8	9	10	11

6, 3, 7, 10, 4, 2, 11, 9, 1, 8, 5

Grading: 1/2 pt for each number in each slot no exceptions, round down so 7 correct is 3 pts.

B1) (10 pts) DSN (Binary Trees)

In a binary tree of n nodes, consider the $n - 1$ paths from the root to each of the other nodes. We define the length of these paths to each equal the number of edges/links that need to be traversed on the shortest path from the root to each of the other nodes. Write a function that takes in a pointer to the root of a binary tree and returns the sum of the lengths of these paths. If the tree has 0 or 1 node, your function should return 0. You may assume that the return value will NOT overflow the int data type. Using the struct below, complete the given function to return this sum. **Notice that for each node, in addition to storing a data value, we store the number of nodes in the subtree rooted at that node. This value in the node is fairly critical for solving this problem efficiently. You may assume this value is always accurate.**

```
typedef struct treenode {
    int data;
    int numNodes;
    struct treenode *left;
    struct treenode *right;
} treenode;

int sumPLen(treenode* root) {

    // 1 pt
    if (root == NULL) return 0;

    // 1 pt to store each answer, 2 pts total.
    int leftRes = sumPLen(root->left);
    int rightRes = sumPLen(root->right);
    int res = leftRes + rightRes;

    // 3 pts total
    if (root->left != NULL) res += root->left->numNodes;

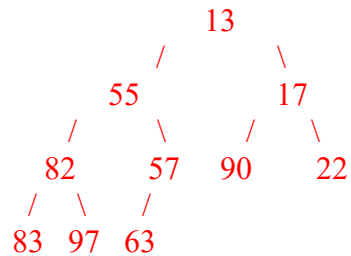
    // 3 pts total
    if (root->right != NULL) res += root->right->numNodes;

    // 1 pt
    return res;
}
```

B2) (5 pts) ALG (Binary Heaps)

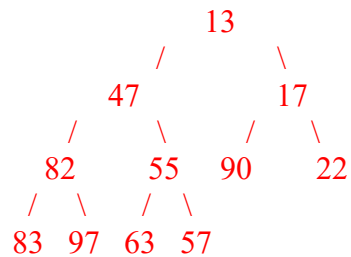
(a) (3 pts) Draw the binary heap that is stored in the array shown below:

index	1	2	3	4	5	6	7	8	9	10
value	13	55	17	82	57	90	22	83	97	63



Grading: 3 pts correct, 2 pts if heap property is maintained but not correct, 0 pts otherwise

(b) (2 pts) Show the resulting binary heap in tree form when 47 is inserted into it.



Grading: 2 pts correct, 1 pt if heap property is maintained with inserted item but answer is wrong, 0 otherwise

B3) (10 pts) DSN (Tries)

Write a **recursive** function that takes in a pointer to the root of a trie and returns the total number of words stored in that trie. Use the struct and function prototype given below.

```
typedef struct trienode {
    int isWord;
    struct trienode* next[26];
} trienode;

int numWords(trienode* root) {

    // 2 pts
    if (root == NULL) return 0;

    // 2 pts
    int res = root->isWord;

    // 1 pt
    for (int i=0; i<26; i++)

        // 4 pts
        res += numWords(root->next[i]);

    // 1 pt
    return res;
}
```

C1) (10 pts) ANL (Algorithm Analysis)

List the **worst case run times** of each of the following operations assuming the efficient implementations of these data structures and algorithms shown in class. Leave your run times in Big-Oh notation in terms of the appropriate variables.

- a) Inserting an item into a queue of **n** items. **$O(1)$**
- b) Removing the minimum item from a binary heap of **n** items. **$O(\lg n)$**
- c) Sorting **n** items using Quick Sort. **$O(n^2)$**
- d) Searching for **k** different items sequentially in an AVL tree of **n** items. **$O(k \lg n)$**
- e) Printing out each subset of **n** items. Assume printing a single item in a single subset takes $O(1)$ time. **$O(n2^n)$**
- f) Inserting **m** items into a linked list that is sorted in order. **$O(m^2)$**
- g) Running a floodfill on a grid with **r** rows and **c** columns. **$O(rc)$**
- h) Running a pre-order traversal on a binary tree of **n** nodes. **$O(n)$**
- i) Returning but not removing the minimum item in a binary heap. **$O(1)$**
- j) Adding a **b** bit binary number to a **c** bit binary number **$O(\max(b,c))$ or $O(b+c)$**

Grading: 1 pt per answer, no exceptions

C2) (5 pts) ANL (Algorithm Analysis)

An algorithm which has a run time of $O(\lg n)$ takes 300 microseconds when it's run on an input with size $n = 10,000$. How long, **in microseconds**, would we expect the algorithm to take when it's run on an input with size $n = 10^9$?

Let $T(n) = c \lg n$ be the run time of the algorithm for input size n . Using the given information, we have

$$T(10^4) = c \lg(10^4) = 300 \text{ microseconds}$$

$$c = \frac{300 \text{ microseconds}}{\lg 10^4} = \frac{300 \text{ microseconds}}{4 \lg 10} = \frac{75 \text{ microseconds}}{\lg 10}$$

Now, let's solve for $T(10^9)$:

$$T(10^9) = \frac{75 \text{ microseconds}}{\lg 10} \times \lg(10^9) = 75 \text{ microseconds} \times \frac{9 \lg 10}{\lg 10} = \mathbf{675 \text{ microseconds}}$$

Grading: 1 pt equation to solve for c , 1 pt solve for c

1 pt plug in 10^9 , 1 pt get unsimplified expression, 1 pt get to final form

C3) (10 pts) ANL (Sums)

Determine a closed form of the following sum in terms of n. Please keep your answer in factored form. Each term in your factored form will either be of the form 2^a or $(2^a + b)^2$, where a and b are either constants or linear expressions in terms of n.

$$\sum_{i=1}^{2^n} (6i^2 + 2i)$$

$$\begin{aligned}\sum_{i=1}^{2^n} (6i^2 + 2i) &= \left(\sum_{i=1}^{2^n} 6i^2 \right) + \left(\sum_{i=1}^{2^n} 2i \right) \\&= \frac{6(2^n)(2^n + 1)(2 \times 2^n + 1)}{6} + \frac{2(2^n)(2^n + 1)}{2} \\&= (2^n)(2^n + 1)(2 \times 2^n + 1) + (2^n)(2^n + 1) \\&= (2^n)(2^n + 1)[2 \times 2^n + 1 + 1] \\&= (2^n)(2^n + 1)[2 \times 2^n + 2] \\&= (2^n)(2^n + 1)2[2^n + 1] \\&= (2^{n+1})(2^n + 1)^2\end{aligned}$$

Grading: 1 pt split sum

2 pts plug in accurately for i^2

1 pt plug in accurately for i

1 pt cancel 6 and 2 in the separate fractions

2 pts factor out $(2^n)(2^n+1)$

1 pt factor out the 2 from $2 \times 2^n + 2$

2 pts to get all the way to the end (1 pt absorb 2, 1 pt square)

D1) (5 pts) DSN (Recursive Coding)

In class we discussed the idea of a "lowest one bit." Traditionally, this means minimum value of each bit set to 1 in a binary number. For example, for 24, which, in binary is 11000, the lowest 1 bit is 8, since the right most bit set to one is in the third bit from the right, counting from 0, so it has the value 2^3 . Consider a different, but related function which returns the place value (power of 2) of the lowest one bit. Write this function recursively below. **You may assume that the input to the function is a positive integer.**

```
int lowOneBitPos(int n) {  
    if (n&1) return 0;           // 2 pts  
    return 1 + lowOneBitPos(n>>1); // 3 pts  
}
```

D2) (10 pts) ALG (Sorting)

(a) (5 pts) Show the result of running the partition algorithm shown in class on the array below. Use 13 as the partition element.

Before	13	6	2	9	7	15	4
After	4	6	2	9	7	13	15

Grading: 5 pts for all 7 items being in the correct spots
3 pts for 5 items being in the correct spots
1 pt for 3 or 4 items being in the correct spots
0 pts otherwise

(b) (5 pts) Show the state of the array below after each iteration of selection sort (where we select for the maximum element).

Initial Array	27	6	19	13	42	9
1 st Iteration	27	6	19	13	9	42
2 nd Iteration	9	6	19	13	27	42
3 rd Iteration	9	6	13	19	27	42
4 th Iteration	9	6	13	19	27	42
5 th Iteration	6	9	13	19	27	42

Grading: 1 pt per row, row has to be perfectly correct to get the point.

D3) (10 pts) ALG (Bitwise Operators)

We define the bit count of an integer to be the number of bits set to 1 in the unsigned binary representation of that number. Write a function that takes in two integers, a and b, and uses a brute force method to determine the sum of the bit counts of each integer in between a and b. You may assume that both a and b are positive with $a < b < 2^{30}$ and that the result will not overflow the int data type.

```
int sumBitsOn(int a, int b) {  
  
    int res = 0;                                // 1 pt  
  
    for (int i=a; i<=b; i++)                    // 2 pts  
  
        for (int j=0; j<30; j++)                // 2 pts  
  
            if (i&(1<<j))                        // 3 pts  
                res++;                            // 1 pt  
  
    return res;                                // 1 pt  
  
}
```